

Clifford-BEM: Ausarbeitung zu AP 3

Teil I: $\mathcal{H}$ -Matrix mit Adaptiver Kreuzapproximation;
kombinierter Löser am Würfel

Arbeitsstand, Version 0.3

Überblick

Grundlage ist der Galerkin-Prototyp aus AP 2 mit stückweise konstanten Multivektor-Dichten auf ebenen Dreiecken. Der Randoperator E_k wird hierarchisch komprimiert (`hmat.py`), und die Formulierung T_1 wird mit GMRES über das komprimierte Matrix-Vektor-Produkt gelöst (`scatter_h.py`).

Ergebnisse.

- Der Fehler des Matrix-Vektor-Produkts folgt der Toleranz: $\approx 0,1 \varepsilon$.
- Bei 3920 Dreiecken (31360 Unbekannte) braucht der Operator 270 MB statt 15 GB für die dichte Matrix.
- Die Streurechnung auf der Kugel ist jetzt bis 23040 Unbekannte möglich. Für Glas ergibt die Extrapolation $Q_{\text{ext}} = 0,21511$ gegenüber Mie 0,21510.
- Die gemeinsame Kreuzapproximation aller Kernkomponenten spart gegenüber der komponentenweisen 10 bis 19 % Speicher bei gleicher Genauigkeit (H3, erster Teil).
- Nahezu lineare Komplexität (H3, zweiter Teil): Im Python-Bereich (bis 3920 Dreiecke) wuchs der Speicher noch wie $N^{1,39}$. Mit dem C++-Kern bis 20480 Dreiecke zeigt sich $\mathcal{O}(N \log^2 N)$ (Abschnitt 6).
- Am Würfel arbeiten Kompression und Kanten-/Eck-Vorkonditionierung zusammen: Die GMRES-Iterationen sind von der Kantenauflösung unabhängig (Abschnitt 5).

1 Konzept

Was komprimiert wird. Für ein Dreieckspaar ist der Galerkin-Block

$$E_{ij} = -\frac{2}{\sqrt{|\tau_i||\tau_j|}} L((s_{ij} + \mathbf{v}_{ij}) \mathbf{n}_j), \quad (s_{ij}, \mathbf{v}_{ij}) = \int_{\tau_i} \int_{\tau_j} \Phi_k(x - y) dS_y dS_x.$$

Gespeichert werden nur die vier Kernkomponenten $K_c(i, j)$, also s und die drei Komponenten von $\mathbf{v}$, nicht die 8×8 -Blöcke. Weil E_{ij} linear in K_c ist, lässt sich das Matrix-Vektor-Produkt schreiben als

$$(Ex)_i = -\frac{2}{\sqrt{|\tau_i|}} \sum_{c=0}^3 \sum_j K_c(i, j) z_j^c, \quad z_j^c = \frac{L(\mathbf{e}_c \mathbf{n}_j) x_j}{\sqrt{|\tau_j|}}, \quad \mathbf{e}_0 = 1.$$

Das spart gegenüber den Blöcken den Faktor 16, schon bevor komprimiert wird.

Clusterbaum und Zulässigkeit. Der Clusterbaum entsteht durch Halbierung entlang der längsten Achse der Schwerpunkte, mit höchstens 32 Dreiecken je Blatt. Ein Blockpaar (τ, σ) ist zulässig, wenn $\min(\text{diam } \tau, \text{diam } \sigma) \leq \eta \text{dist}(\tau, \sigma)$ mit $\eta = 1$ gilt. Außerdem muss $\text{dist} > 3h_{\max}$ sein, damit alle Paare im Fernfeld der Gauß-Regel liegen, und $k \text{diam} < 20$. Unzulässige Blätter werden dicht mit der Nahfeldbehandlung aus AP 2 assembliert; diese ist jetzt über die Nachbardreiecke vektorisiert.

Kreuzapproximation. Verwendet wird ACA mit partieller Pivotisierung und Abbruch bei $\|u_k\| \|v_k\| \leq \varepsilon \|S_k\|_F$, danach eine QR/SVD-Nachkompression. Bei komplexen Faktoren muss es in der Nachkompression $V = Q_V Z^T$ heißen und nicht $Q_V Z^H$; die konjugierte Variante erzeugte Blockfehler von 20 bis 75 %.

Zwei Varianten.

- *komponentenweise*: vier getrennte ACAs für die vier Kernkomponenten. Speicher $\sum_c r_c(|\tau| + |\sigma|)$.
- *gemeinsam*: eine ACA auf der gestapelten Matrix $|\tau| \times 4|\sigma|$. Die Zeilenfaktoren und Pivots werden also über die Grade geteilt. Speicher $r(|\tau| + 4|\sigma|)$.

Eine echte Block-ACA mit 8×8 -Pivotblöcken ist noch nicht umgesetzt.

2 Genauigkeit und Variantenvergleich

Kugel mit 1280 Dreiecken, $k = 1,5$. Der Fehler ist der relative Fehler des Matrix-Vektor-Produkts gegenüber 40 exakt assemblierten Zeilen (`test_hmat_large.py`):

ε	Fehler	gemeinsam		komponentenweise		
		Niedrigrang	Aufbau	Fehler	Niedrigrang	Aufbau
10^{-3}	$9,4 \cdot 10^{-5}$	18,8 MB	35 s	$9,5 \cdot 10^{-5}$	20,9 MB	53 s
10^{-4}	$1,1 \cdot 10^{-5}$	25,1 MB	37 s	$9,8 \cdot 10^{-6}$	31,1 MB	62 s
10^{-5}	$1,1 \cdot 10^{-6}$	35,0 MB	40 s	$1,0 \cdot 10^{-6}$	41,8 MB	72 s

Bei gleicher Genauigkeit spart die gemeinsame Variante 10, 19 bzw. 16 % Speicher in den Niedrigrangblöcken. Sie ist beim Aufbau schneller, und ihr Matrix-Vektor-Produkt ebenfalls (0,14 s gegenüber 0,20–0,24 s). Das stützt den ersten Teil von H3 in mäßigem Umfang. Die dichten Nahblöcke (29 MB) sind bei beiden Varianten gleich.

3 Skalierung

Gemeinsame Variante, $\varepsilon = 10^{-4}$, $k = 1,5$:

N	Unbekannte	$\mathcal{H}$ gesamt	davon Niedrigrang	dichter Kern	Aufbau	Mat-Vek
720	5 760	25,6 MB	4,7 MB	32 MB	17 s	0,07 s
1280	10 240	53,7 MB	25,1 MB	100 MB	37 s	0,14 s
2000	16 000	114,4 MB	37,8 MB	244 MB	55 s	0,25 s
2880	23 040	166,7 MB	89,9 MB	506 MB	93 s	0,35 s
3920	31 360	269,9 MB	120,9 MB	938 MB	124 s	0,53 s

Die dichte E -Matrix mit 8×8 -Blöcken hätte bei $N = 3920$ etwa 15 GB. Fehler in allen Fällen $0,85$ bis $1,1 \cdot 10^{-5}$.

Der Kompressionsfaktor gegenüber dem dichten Kern steigt von 1,2 auf 3,5. Eine Ausgleichsgerade ergibt Speicher $\propto N^{1,39}$ und Aufbauzeit $\propto N^{1,18}$; Speicher/ $(N \log N)$ steigt noch leicht an. Der Bereich ist also präasymptotisch: Mit wachsendem N werden immer größere Blöcke zulässig, und die strenge Bedingung $\text{dist} > 3h$ hält viele Blätter dicht (bei $N = 3920$ sind 55 % des Speichers dichte Nahblöcke). Nahezu lineare Komplexität (H3, zweiter Teil) ist damit weder gezeigt noch widerlegt. Nötig sind größere N und eine Abstimmung von Blattgröße und Zulässigkeit.

4 Streuung jenseits der dichten Grenze

Gelöst wird $T_1 h = h^{\text{inc}}$ mit GMRES (Neustart 200, Toleranz 10^{-6}) und der punktweisen Vorkonditionierung $2(1 + J)^{-1}$. Beide Operatoren E_{k_1} und E_{k_2} sind als $\mathcal{H}$ -Matrizen gespeichert. Die Werte für $N \leq 500$ stammen aus der dichten Rechnung in AP 2.

N	Unbekannte	Glas, $\omega a = 1$		Gold, $\omega a = 0,5$	
		Q_{ext}	GMRES	Q_{ext}	GMRES
500 (dicht)	4 000	0,20721	12	0,64732	35
720	5 760	0,20959	10	0,63429	26
1280	10 240	0,21198	10	0,61987	25
2000	16 000	0,21310	10	0,61226	25
2880	23 040	0,21371	10	0,60764	25
extrapoliert ($N = 1280, 2000, 2880$)		0,21511		0,59247	
Mie		0,21510		0,59002	

Die GMRES-Zahlen der dichten Zeile bei $N = 500$ sind ohne Neustart und mit Toleranz 10^{-8} ermittelt und daher nicht direkt vergleichbar. Mit $\mathcal{H}$ -Matrizen und gleicher Einstellung sind die Iterationszahlen von N unabhängig.

Für Glas ist die Konvergenz sauber zweiter Ordnung ($p = 1,99$), und die Extrapolation trifft Mie auf $4 \cdot 10^{-5}$. Für Gold ist die beobachtete Ordnung $p \approx 1,46$ geringer, und die Extrapolation liegt 0,4 % über Mie. Die Ursache ist noch offen. Plausibel sind die Kanten des Polyeders, auf die plasmonische Antworten empfindlich reagieren, sowie die langsam konvergente Nahfeldquadratur (AP 2). Die Komprimierung ist nicht die Ursache, denn ihr Fehler liegt bei 10^{-5} .

5 Würfel: Kompression und Kanten-/Eck-Vorkonditionierung kombiniert

Netz. Damit die Kantenblöcke aus AP 2.5 nicht entlang der Kante wachsen, wird jede Seitenfläche in drei Zonen zerlegt (`cube_aniso.py`): ein grobes Innenquadrat (4×4), vier Kantenstreifen, die nur *quer* zur Kante geometrisch gradiert sind (Breite 0,5, L Schichten, entlang der Kante 4 Elemente), und vier Eckquadrate, die in beiden Richtungen gradiert sind. An den Zonengrenzen entstehen hängende Knoten; für stückweise konstante L^2 -Ansätze ist das zulässig. Die Spurtrennung gilt auf dem nichtkonformen Netz mit 2 bis 7 % Fehler bei 432 Dreiecken. Das ergibt 768, 1200, 1728 und 2352 Dreiecke für $L = 2, \dots, 5$.

Löser. E_{k_1} und E_{k_2} werden als $\mathcal{H}$ -Matrizen gespeichert ($\varepsilon = 10^{-4}$, ohne Symmetriezerlegung). Die Kanten- und Eckblöcke (Radius $R = 0,25$) werden exakt mit Nahfeld assembliert und dicht per LU invertiert. Außerhalb der Blöcke wird $(\frac{1}{2}(1 + J))^{-1}$ verwendet. GMRES läuft rechtsvorkonditioniert, mit Neustart 300 und Toleranz 10^{-6} (`cube_h.py`), bei $\omega a = 0,5$.

L	N	$\kappa = -3 + 0,3i$			Gold $-11 + 1,2i$		
		ohne	Blöcke	σ_{ext}	ohne	Blöcke	σ_{ext}
2	768	73	49	18,235	64	45	19,909
3	1200	84	50	18,686	67	47	19,651
4	1728	96	49	18,593	69	47	19,404
5	2352	106	49	18,326	71	47	19,243

$\mathcal{H}$ -Speicher beider Operatoren: 50, 99, 169, 254 MB. Größter Block: 192, 416, 608, 800 Unbekannte. Nur mit $2(1 + J)^{-1}$: 66 bzw. 75 ($-3 + 0,3i$) und 57 bzw. 61 (Gold) für $L = 2, 3$. σ_{ext} mit und ohne Vorkonditionierung auf 6 Stellen gleich.

Befunde.

- *Iterationen:* Mit Kanten- und Eckblöcken sind sie konstant (49 bzw. 47), auch mit komprimierten Operatoren und über alle Symmetriesektoren zugleich. Ohne Blöcke wachsen sie um etwa 11 ($-3 + 0,3i$) bzw. 2–3 (Gold) je Stufe. Damit ist AP 2.5 mit AP 3 kombiniert.
- *Genauigkeit:* Für Gold fällt σ_{ext} monoton (Differenzen 0,26; 0,25; 0,16). Für $-3 + 0,3i$ schwankt der Wert (18,24 $\rightarrow$ 18,69 $\rightarrow$ 18,59 $\rightarrow$ 18,33). Das entspricht qualitativ der Vorhersage aus AP 1: Gold liegt außerhalb des Resonanzfensters, $-3 + 0,3i$ nahe daran. Da nur die Kantenzonen verfeinert werden (Innen- und Längsauflösung sind fest), konvergiert die Folge gegen die Lösung eines Netzes mit fester grober Restauflösung, nicht gegen die exakte Lösung. Ein Referenzwert für den Würfel fehlt.
- *Blockanteil:* Bei $L = 5$ decken die Blöcke 82% der Unbekannten ab, weil in diesem Netz fast alle Elemente in den verfeinerten Zonen liegen. Die Blöcke selbst sind klein (höchstens 800 Unbekannte), die Vorkonditionierung ist hier aber nahezu ein blockweiser Direktlöser. Bei feinerem Innenbereich sinkt der Anteil. Eine $\mathcal{H}$ -LU der Blöcke (AP 3.4 im engeren Sinn) war bei diesen Größen nicht nötig und ist nicht umgesetzt.
- *Aufbauzeit:* Bei $L = 5$ etwa 215 s je Wellenzahl, mit 3216 dichten gegenüber 2590 Niedrigrangblöcken. Ursache ist die Zulässigkeitsbedingung $\text{dist} > 3h_{\text{max}}$: Die gestreckten Kantendreiecke haben lange Kanten (0,25) und machen viele Blöcke unzulässig. Nötig ist eine Nahfeldquadratur, die auch für gestreckte Elemente in kleinerem Abstand genau ist, oder eine Zulässigkeit nach der kurzen Elementausdehnung mit angepasster Fernfeldquadratur.

6 Asymptotik mit dem C++-Rechenkern

Der Kern aus dem Repository `PhotonPower/CliffordBem` (v0.1) implementiert dieselben Kernkomponenten, Nahfeldregeln und ACA-Varianten. Die Einträge stimmen mit dem Prototyp auf 10^{-13} überein, und er ist etwa 6-mal schneller. Einstellungen wie oben ($\varepsilon = 10^{-4}$, gemeinsame ACA). Details und Rohdaten: `docs/results_compression.md` im Repository.

Geometrie	Dreiecke	Unbekannte	Speicher	$10^3 \cdot \text{MB}/(N \log^2 N)$	Aufbau
Kugel, $k = 1,5$	1 280	10 240	55 MB	0,83	6 s
	5 120	40 960	340 MB	0,91	33 s
	11 520	92 160	959 MB	0,95	80 s
	20 480	163 840	1 881 MB	0,93	156 s
Würfel (gleichm.), $k = 1,5$	1 200	9 600	51 MB	0,84	7 s
	6 912	55 296	488 MB	0,90	53 s
	19 200	153 600	1 651 MB	0,88	171 s
Kugel, $k = n/4$	1 280	10 240	55 MB	0,83	6 s
	20 480	163 840	2 075 MB	1,03	192 s

- Der Quotient $\text{Speicher}/(N \log^2 N)$ ist für Kugel und Würfel ab etwa 3000 Dreiecken innerhalb von $\pm 7\%$ konstant. Die Aufbauzeit wächst mit Exponent 1,1 bis 1,2. Die frühere Beobachtung $N^{1,39}$ (Abschnitt 3) war präasymptotisch.
- Kanten und Ecken ändern das Wachstum nicht.
- Bei fester Auflösung je Wellenlänge (etwa 10 Elemente) kostet die Frequenz bis $kD \approx 16$ nur etwa 10 % Speicher. Bei festem Netz wächst der Niedrigrang-Speicher bis $kD = 48$ um 62 %, und der mittlere Rang steigt von 7,0 auf 9,0.
- Gemeinsame gegen komponentenweise ACA: etwa 19 % weniger Niedrigrang-Speicher, konstant über $N = 1280$ bis 11 520, bei halber Aufbauzeit.
- Zu den Kanten gradierte Tensornetze sind dicht-dominiert (bei 3072 Dreiecken 84 % dichte Nahblöcke, 293 MB gegenüber 170 MB gleichmäßig). Das liegt an den auf $h_{\max}$ bezogenen Nahfeld- und Zulässigkeitskriterien und ist mit einer Fernfeldquadratur für gestreckte Elemente zu beheben.

Damit ist H3 (zweiter Teil) im Bereich bis etwa 160 000 Unbekannte und $kD \lesssim 16$ numerisch gestützt: Der Aufwand ist $\mathcal{O}(N \log^\alpha N)$ mit $\alpha \approx 2$ für den Speicher.

7 Einordnung und nächste Schritte

- *H3, erster Teil:* Gemeinsame Pivotisierung über die Grade spart 10–19 %. Eine echte Block-ACA mit 8×8 - bzw. Multivektor-Pivots (AP 3.2) kann das verbessern; zu testen.
- *H3, zweiter Teil:* Mit dem C++-Kern bis 163 840 Unbekannte gestützt: $\mathcal{O}(N \log^2 N)$ für Kugel und Würfel (Abschnitt 6). Offen sind stark gradierte Netze (Kriterien für gestreckte Elemente) und elektrisch große Streuer ($kD \gg 50$).
- *Laufzeit:* Der C++-Kern ist etwa 6-mal schneller als der Python-Prototyp (ein Rechenkern); die Assemblierung ist für OpenMP vorbereitet.
- *AP 3.4:* Kompression und Kanten-/Eck-Vorkonditionierung sind am Würfel kombiniert (Abschnitt 5). Offen sind die $\mathcal{H}$ -LU für große Blöcke und eine Zulässigkeit, die gestreckte Kanten-dreiecke nicht bestraft.